Predicting User Sentiment in Software Issue Reports

ST2MLE — Machine Learning for IT Engineers

A pipeline to classify issue-report sentiment: negative · neutral · positive

Comparing vectorisation strategies and classifiers, with full tuning, imbalance handling, dimensionality reduction, pruning and early stopping.

Problem & objectives

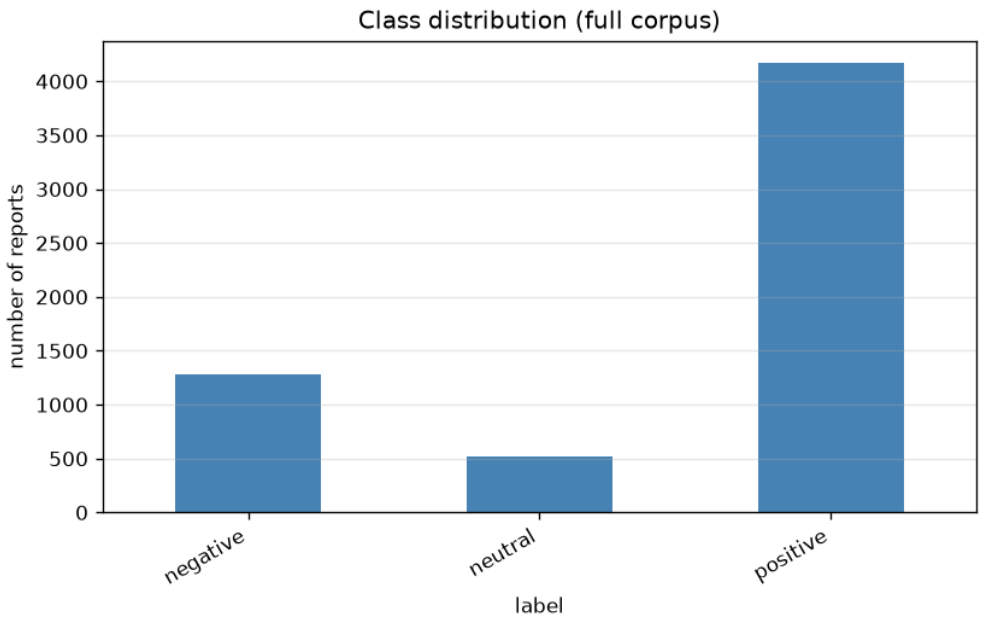
- **Goal:** automatically classify the sentiment of user-reported software issues (bug reports, feature requests, reviews) into **negative / neutral / positive**.
- **Why it matters:** triage angry reports faster, spot satisfaction trends, prioritise fixes.
- **What we compare:**
- 4 vectorisers — Bag-of-Words, TF-IDF, Word2Vec, pre-trained GloVe (+ BERT code)
- 5 classifiers — Naive Bayes, Decision Tree, Random Forest, AdaBoost, Logistic Regression
- **And we study:** class imbalance (SMOTE), PCA, tree pruning, early stopping.

Dataset — real app reviews

- **Real Google Play reviews** (`sealuzh/user_quality`): 288 k reviews, 395 apps.
- **Stars → sentiment:** 1–2★ negative · 3★ neutral · 4–5★ positive.
- Reproducible **stratified sample of 6 000**, keeping the natural skew:

negative	neutral	positive
~21 %	~9 %	~70 %

- **Genuinely hard:** star labels are a proxy, text is short/noisy/informal, 3★ "neutral" overlaps its neighbours. (Synthetic generator bundled as fallback.)



Pipeline overview

`raw text → preprocess → vectorise → [balance] → [reduce] → classify → evaluate`

- **Preprocess:** lower-case, strip punctuation/URLs, tokenise, drop stop-words (**keep negations**), WordNet lemmatise.
- **Vectorise:** BoW · TF-IDF · Word2Vec · pre-trained GloVe.
- **Balance (train only):** SMOTE / under-sampling via `imblearn`.
- **Reduce:** TruncatedSVD (PCA for sparse text).
- **Classify & tune:** 5 models, 5-fold CV, `GridSearchCV`.
- **Evaluate:** accuracy, macro P/R/F1, confusion matrix.

Engineering: preprocess once + cache vectorisers → whole study in ~8 min.

Preprocessing

- **Clean:** lower-case → remove URLs/HTML/punctuation/digits.
- **Tokenise** then **remove stop-words** — but **keep negations** (`not`, `no`, `never`…) because they flip sentiment.
- **Lemmatise** (WordNet, verb-then-noun): crashing → crash, issues → issue.
- Implemented as a stateless scikit-learn transformer → no train/test leakage.

Example `"Critical issue -- The app keeps crashing!!!"` → `critical issue app keep crash`

Vectorisation — 4 strategies

Strategy	Idea	Output
Bag-of-Words	n-gram counts	sparse, ≥ 0
TF-IDF	counts × inverse document frequency	sparse, ≥ 0
Word2Vec	mean of embeddings trained on our corpus	dense
GloVe (pre-trained)	mean of Wikipedia/Gigaword 100-d vectors	dense

- Sparse + non-negative → feeds **MultinomialNB**; dense → **GaussianNB**.
- **BERT** vectoriser is implemented & documented; needs the model hub (unavailable here) so it is skipped automatically.

Models & tuning

- **Naive Bayes** — `MultinomialNB` (sparse) / `GaussianNB` (dense), auto-selected.
- **Decision Tree** — pre-pruning (`max_depth`, `min_samples_leaf`) **and** post-pruning (`ccp_alpha`).
- **Random Forest** — bagged trees.
- **AdaBoost** — boosted stumps.
- **Logistic Regression** — demonstrates **L2 regularisation** (tuned `C`).

All tuned with **`GridSearchCV`, 5-fold cross-validation**, scoring **macro-F1**.

Experimental setup

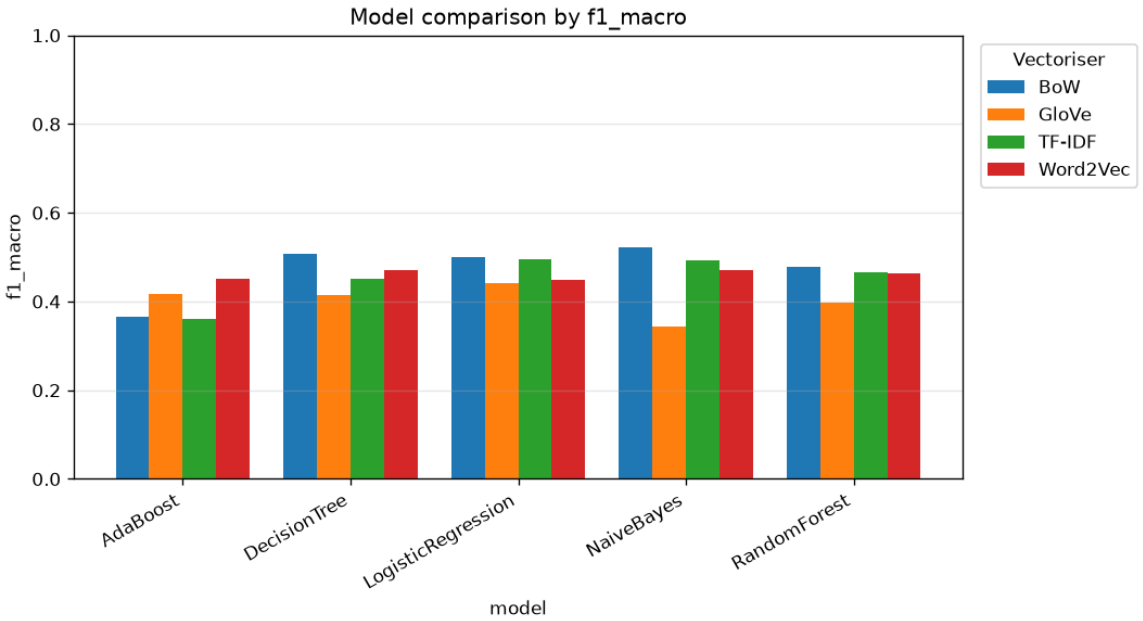
- **Split:** stratified 80 % train / 20 % test.
- **CV:** 5-fold `StratifiedKFold` on the training set.
- **Tuning target:** macro-F1 (weights every class equally on imbalanced data).
- **Reproducibility:** single seed (42); deterministic — a clean re-run reproduces every number byte-for-byte.
- **Compute:** 4 cores; full study ≈ 8 min thanks to cached vectorisers.

Results — model comparison

Best classifier per vectoriser (test macro-F1):

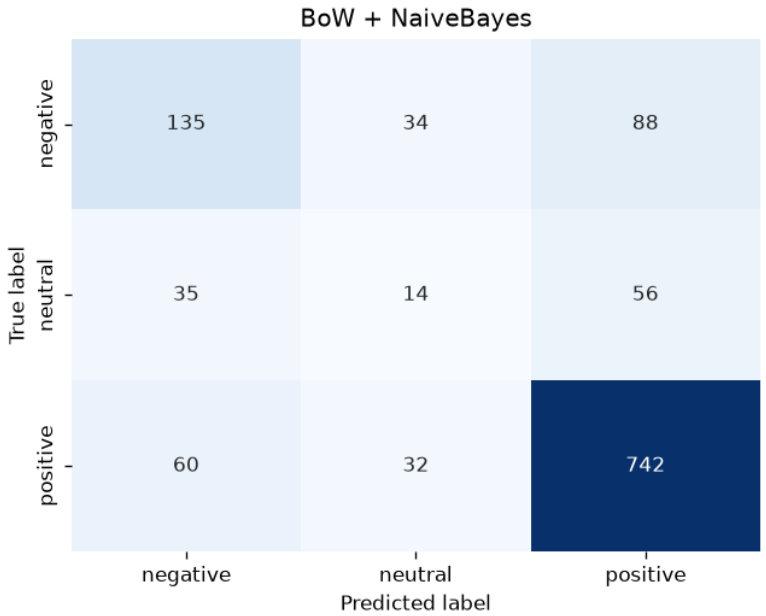
Vectoriser	Best classifier	Macro-F1
BoW	**Naive Bayes**	**0.523**
TF-IDF	Logistic Regression	0.494
Word2Vec	Decision Tree	0.471
GloVe (pre-trained)	Logistic Regression	0.440

Macro-F1 ≈ 0.45–0.52 (real, noisy data). AdaBoost & GloVe weakest. Full 20-row table in the report.



Results — best model

BoW + Naive Bayes — macro-F1 0.523, accuracy 0.745 — confusion matrix on the held-out test set:



- High accuracy is driven by the dominant **positive** class.
- The minority **neutral** (3★) class is hardest — it overlaps both neighbours.

Class-imbalance handling

TF-IDF + Logistic Regression; only the resampling strategy changes (train folds only).

Strategy	Accuracy	Macro-F1	Recall neu
none	**0.772**	0.482	**0.010**
SMOTE	0.640	**0.502**	**0.295**
under-sampling	0.601	0.485	0.390

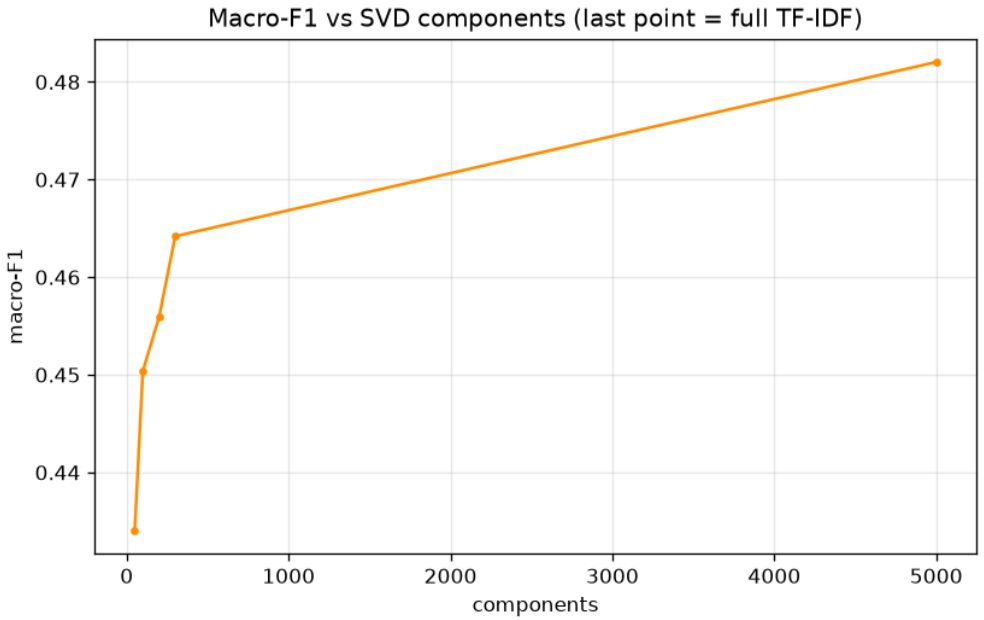
Takeaway — the clearest result: the baseline gets 0.77 accuracy by ignoring neutrals (recall **0.01**). SMOTE lifts neutral recall **~30×** and improves macro-F1 → **accuracy is a trap; resampling rescues the minority classes.**

Dimensionality reduction (PCA / TruncatedSVD)

TF-IDF (5 000 features) → TruncatedSVD; downstream macro-F1 (Logistic Regression):

Components	Variance	Macro-F1
100	29 %	0.450
300	47 %	**0.464**
5 000 (full)	100 %	0.482

Takeaway: 300 components (6 % of features) keep ~96 % of full performance — a ~17× compression, and essential before dense models (e.g. boosting).

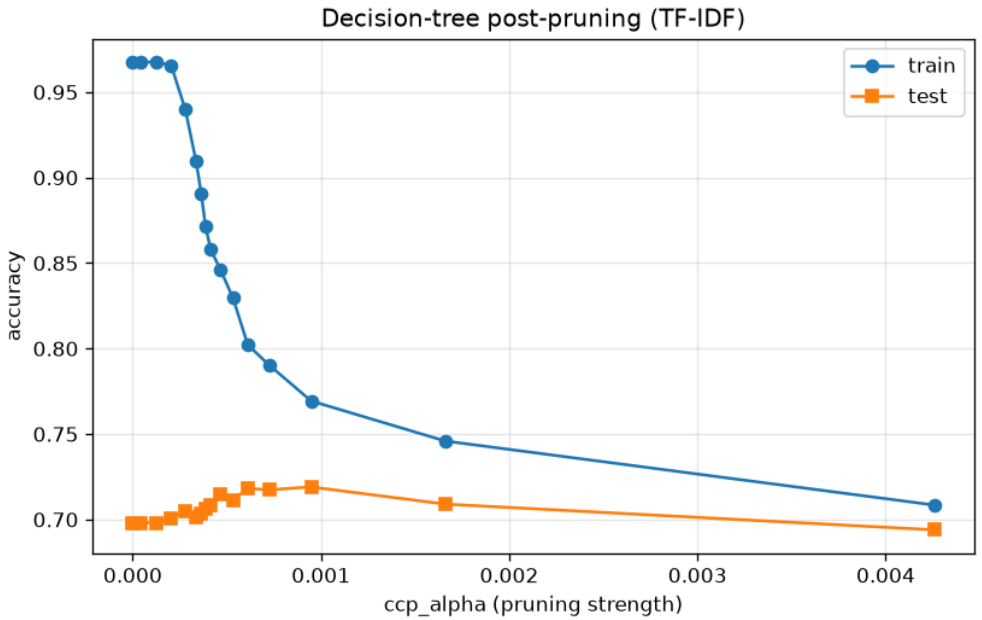


Decision-tree pruning

Cost-complexity (`ccp_alpha`) post-pruning on TF-IDF:

Tree	# nodes	Train acc	Test acc
unpruned	2 083	0.968	0.698
best pruned	**95**	0.769	**0.719**
over-pruned	13	0.708	0.694

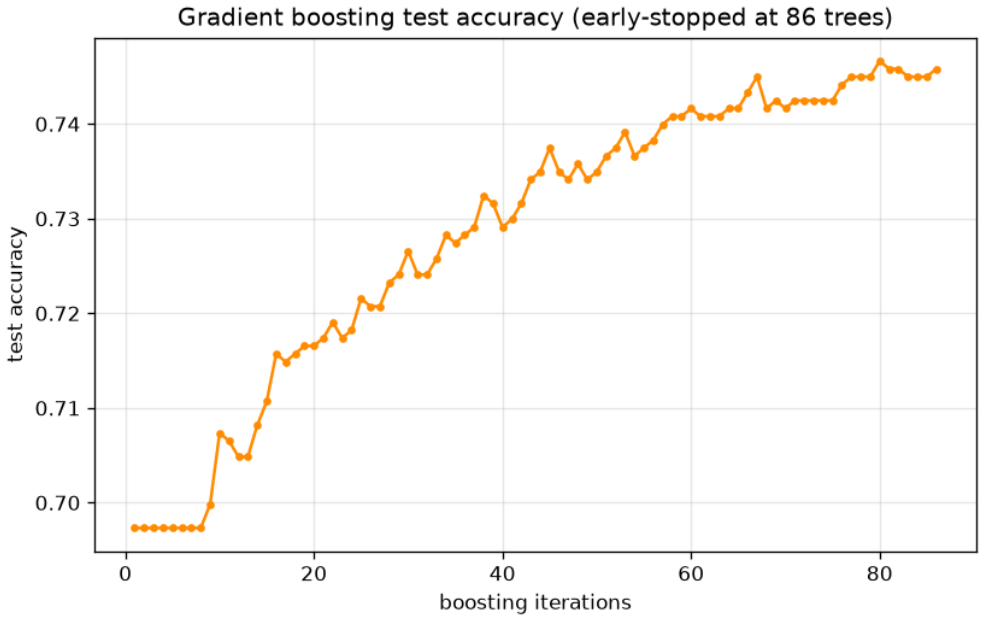
Takeaway: pruning cuts 2 083 → 95 nodes, **raises test accuracy 0.70 → 0.72**, and shrinks the over-fitting gap 0.27 → 0.05 — textbook bias–variance.



Early stopping (gradient boosting)

TF-IDF → SVD(100) → Gradient Boosting, ceiling 500 trees, `n_iter_no_change=10`.

- **Stopped after 86 of 500 trees** — ~6× less training, no loss of test quality.
- Validation monitoring halts once added trees stop helping → guards over-fitting.



Discussion — trade-offs

- **Simple sparse models win** on short, noisy reviews — BoW + Naive Bayes (0.523).
- **Accuracy is a trap** at 70 % positive; **macro-F1** is the metric that matters.
- **AdaBoost** collapses on sparse high-dim features (stumps see 1 word of ~5 000); **pre-trained GloVe** is weakest — mean-pooling washes out lexical cues.
- **SMOTE** rescues minority recall; **PCA** buys efficiency; **pruning & early stopping** both curb over-fitting.

Challenges & future work

Challenges - Kaggle/HF & GitHub API blocked → sourced a **real** app-review corpus from a raw GitHub URL; star ratings are an imperfect (proxy) sentiment label. - Severe imbalance + fuzzy neutral class → low macro-F1 despite high accuracy. - `MultinomialNB` rejects negatives → auto-switch to `GaussianNB` for embeddings.

Future work - Add **GitHub/JIRA issues** directly once API access is available. - Enable **BERT** / fine-tuned transformers (code already provided). - **Class-weighting** and cost-sensitive thresholds as further imbalance remedies.

Conclusion

- Built a **complete, reproducible** sentiment pipeline on a **real app-review** dataset.
- Compared **4 vectorisers × 5 classifiers**, fully tuned with 5-fold CV.
- **Best: BoW + Naive Bayes (macro-F1 0.523)**; SMOTE was decisive for the minorities.
- Demonstrated imbalance handling, PCA, pruning and early stopping end to end — with an honest account of why real-data scores are modest.

Thank you — questions?